# Trainable, Differentiable SPPT

A student's guide to learning the parameters of a stochastic weather model by gradient descent

Stochastic parametrization tutorial
`examples/stochastic_sppt/`

July 16, 2026

**Abstract**

This document is a self-contained tutorial for a reader who has taken a first course in atmospheric dynamics and knows some Python, but who has *not* necessarily seen ensemble forecasting, proper scoring rules, or stochastic gradient estimation. We build a stochastic version of an idealised (Held–Suarez) atmospheric model using **SPPT** — Stochastically Perturbed Parametrization Tendencies — and then *learn* the three parameters that control the stochasticity by **gradient descent through the model**. Because the entire model is written as a differentiable `JAX` program, we can compute the derivative of an ensemble forecast score with respect to the noise parameters and optimise them. We explain every ingredient from first principles: the perturbation scheme, the spectral pattern generator, how "truth" data is generated and stored, the scoring rule (CRPS and its finite-ensemble corrections), the calibration diagnostics (spread–error ratio, rank histogram), and — in detail — *how one differentiates through randomness and what the gradient is taken with respect to*. We show the results of the first milestone (parameter recovery), discuss honestly where and why the recovered value departs from the truth, and give exercises and further reading.

## Contents

**1 Why a weather model needs randomness** **2**

**2 The deterministic core** **2**

**3 The stochastic pattern generator** **3**
3.1 A first-order autoregressive process, one coefficient at a time . . . . . 3
3.2 Setting the amplitude and the length scale . . . . . 4

**4 Generating and storing the "truth"** **5**

**5 Scoring an ensemble: the CRPS** **6**

**6 Is the ensemble calibrated? Two diagnostics** **7**
6.1 Spread–error ratio . . . . . 7
6.2 Rank histogram (Talagrand diagram) . . . . . 8

**7 Learning the parameters: differentiating through randomness** **8**
7.1 The reparameterization (pathwise) gradient . . . . . 8
7.2 Common random numbers . . . . . 9
7.3 Reverse mode, checkpointing, and why we train at T21 . . . . . 9
7.4 Why $\sigma$ settles slightly *above* 0.5 . . . . . 10

**8 Experimental protocol** **10**

**9 Results — Milestone M1** **11**
9.1 Parameter recovery . . . . . 11
9.2 The recovered $\sigma$ gives a calibrated ensemble . . . . . 11

**10 How the figures were made** **12**

**11 Limitations and open questions** **13**

**12 Exercises** **13**

**Further reading** **14**

# 1 Why a weather model needs randomness

A weather or climate model integrates the equations of motion on a grid. Motions smaller than the grid box — convection, turbulence, cloud microphysics, gravity waves — cannot be resolved, so their *average* effect on the resolved flow is supplied by *parametrizations*: deterministic formulas that return a tendency (a rate of change $\partial X/\partial t$) for each prognostic variable $X$. These formulas are approximations with real, structural uncertainty: two defensible convection schemes disagree, and neither is "correct".

A single deterministic forecast hides this uncertainty. An **ensemble forecast** exposes it: run the model many times with slightly different initial conditions and/or slightly perturbed physics, and the *spread* of the resulting forecasts is an estimate of the forecast uncertainty. For the ensemble to be trustworthy it must be **reliable** (also called *calibrated*): on the occasions when the ensemble is confident (small spread) it should usually be right, and when it is uncertain (large spread) the truth should more often be far from the mean. Quantifying and achieving reliability is the technical heart of this tutorial.

**SPPT** (Palmer et al., 2009) represents model uncertainty by multiplying the *total parametrized physics tendency* by a smoothly varying random field. If the deterministic physics returns a tendency $P_X$ for variable $X$, SPPT replaces it with

$$\tilde{P}_X(\mathbf{x}, z, t) \;=\; \big(1 + \mu(z)\, r(\mathbf{x}, t)\big)\, P_X(\mathbf{x}, z, t), \tag{1}$$

where $r(\mathbf{x}, t)$ is a random pattern that is smooth in space and correlated in time, $\mu(z) \in [0, 1]$ is a vertical taper (SPPT is switched off near the surface and in the stratosphere, where the multiplicative assumption is poor), and the same $r$ multiplies the wind and temperature tendencies at a given column so that the perturbation is physically consistent. Different draws of $r$ give different ensemble members.

**The idea of this project.** The pattern $r$ is controlled by three numbers: its amplitude $\sigma$, its temporal decorrelation time $\tau$, and its spatial correlation length $\ell$. Normally these are hand-tuned. Here the model is *differentiable*, so we instead **learn** $(\sigma, \tau, \ell)$ by minimising an ensemble forecast score with gradient descent. Reproducing SPPT and the score faithfully is standard; the *trainable / differentiable* framing is the contribution.

# 2 The deterministic core

The host model is a dry primitive-equation spectral dynamical core forced by the Held & Suarez (1994) benchmark: Newtonian relaxation of temperature towards a prescribed radiative-equilibrium profile and Rayleigh damping of low-level winds. Held–Suarez is a standard idealisation — it produces a realistic midlatitude jet and baroclinic eddies without any moisture

or radiation code — so it is the perfect laboratory for a methods paper: cheap, reproducible, and free of confounding physics. In the code the Held–Suarez forcing plays the role of "the parametrized physics" $P_X$ that SPPT perturbs (`held_suarez.py`).

The dynamics are *spectral*: the prognostic fields are represented by their spherical-harmonic coefficients and truncated triangularly at total wavenumber $T$ ("T21" means $T = 21$). We run at several resolutions (Table 1); the differentiable training in this tutorial is done at **T21** for reasons explained in §7.3.

Table 1: Resolutions (`config.py`). $L$ is the spherical-harmonic bandlimit ($n_{\text{lat}} = L$, $n_{\text{lon}} = 2L - 1$); $T$ is the triangular truncation; $\Delta t$ is the time step (chosen for numerical stability).

| name | $L$ | $T$ | $\Delta t$ (s) |
|---|---|---|---|
| T21 | 32 | 21 | 1800 |
| T42 | 64 | 42 | 1200 |
| T85 | 128 | 85 | 600 |
| T127 | 192 | 127 | 450 |

Everything — dynamics, physics, SPPT, and the score — is a pure `JAX` function, so it can be composed with `jax.jit` (compile), `jax.vmap` (vectorise over ensemble members), and, crucially, `jax.grad` (automatic differentiation). That last property is what makes training possible.

**Float64 is mandatory.** Spectral dynamics are numerically stiff; single precision silently corrupts them. Every entry point forces `JAX_ENABLE_X64=True`. On Apple Silicon, `JAX` defaults to a Metal backend that *cannot* do float64, so we also pin `JAX_PLATFORMS=cpu` (a cluster's CUDA setting still wins). If you forget this, you will get either a crash or, worse, quietly wrong numbers.

# 3 The stochastic pattern generator

The pattern $r$ is built in *spectral* space, because there its statistics (variance and correlation length) are set analytically. This follows Appendix 8.1 of Palmer et al. (2009).

## 3.1 A first-order autoregressive process, one coefficient at a time

Each spherical-harmonic coefficient $r_{\ell m}$ (degree $\ell$, order $m$) evolves as an independent complex **AR(1)** (first-order autoregressive) process:

$$r_{\ell m}^{(t+\Delta t)} \;=\; \phi\, r_{\ell m}^{(t)} \;+\; \sigma_\ell\, \eta_{\ell m}^{(t)}, \qquad \phi = e^{-\Delta t/\tau}, \tag{2}$$

where $\eta_{\ell m}^{(t)}$ are i.i.d. complex standard Gaussians (real and imaginary parts $\sim \mathcal{N}(0,1)$), and $\sigma_\ell$ is a forcing amplitude that depends only on the total wavenumber $\ell$. Equation (2) is the discrete analogue of an Ornstein–Uhlenbeck process: $\phi$ close to 1 (large $\tau$) means the pattern changes slowly; $\phi$ close to 0 (small $\tau$) means it is refreshed every step.

**Stationary variance.** Taking the variance of (2) in the stationary state ($\mathrm{Var}[r_{\ell m}^{(t+\Delta t)}] = \mathrm{Var}[r_{\ell m}^{(t)}] =: v_\ell$) and using independence of $\eta$ from $r$,

$$v_\ell = \phi^2 v_\ell + \sigma_\ell^2 \underbrace{\mathrm{Var}[\eta]}_{=1} \quad\Longrightarrow\quad v_\ell = \frac{\sigma_\ell^2}{1-\phi^2}. \tag{3}$$

This is why the code *initialises* the coefficients from $r_{\ell m}^{(0)} = (1-\phi^2)^{-1/2}\,\sigma_\ell\,\eta_{\ell m}$: it starts the process already in its stationary distribution, so there is no spin-up transient.

### 3.2 Setting the amplitude and the length scale

The forcing amplitude has a shape in wavenumber and an overall scale:

$$\sigma_\ell = F_0 \exp\!\Big(-\tfrac{1}{2}\,\kappa_T\,\ell(\ell+1)\Big), \qquad \kappa_T = \frac{1}{2}\left(\frac{\ell_{\mathrm{corr}}}{a}\right)^2, \tag{4}$$

where $a$ is the planetary radius and $\ell_{\mathrm{corr}}$ is the physical correlation length. Because $\ell(\ell+1)$ is the eigenvalue of the Laplacian on the sphere for degree $\ell$, the factor $\exp(-\frac{1}{2}\kappa_T\,\ell(\ell+1))$ is a Gaussian low-pass filter: large $\ell_{\mathrm{corr}}$ (large $\kappa_T$) suppresses high wavenumbers and yields a spatially *smooth* pattern with big features; small $\ell_{\mathrm{corr}}$ lets small scales through.

The overall scale $F_0$ is fixed by demanding that the *grid-point* variance of $r$ equals the target $\sigma^2$. Summing the stationary variance (3) over all coefficients (with the $2\ell+1$ multiplicity of order $m$) and matching gives, up to the transform's normalisation convention,

$$F_0 = \sqrt{\frac{C\,\sigma^2\,(1-\phi^2)}{2\sum_{\ell\geq 1}(2\ell+1)\,e^{-\kappa_T\,\ell(\ell+1)}}}, \tag{5}$$

where $C$ is a single scalar (`_VAR_NORM`) reconciling the analytic formula with the particular inverse-transform convention of the spherical-harmonic library. In code (`sppt.py`):

```
def sigma_n(params, trans_config, dt):
    L, radius = trans_config.L, trans_config.radius
    n = jnp.arange(L) # total wavenumber l
    kappa_t = (jnp.exp(params.log_len) / radius) ** 2 / 2.0
    shape = jnp.exp(-kappa_t * n * (n + 1) / 2.0) # eq. (3): the l-shape
    var_r = jnp.exp(2.0 * params.log_sigma) # sigma^2
    phi = jnp.exp(-dt / jnp.exp(params.log_tau))
    denom = 2.0 * jnp.sum((2*n[1:]+1) * jnp.exp(-kappa_t * n[1:]*(n[1:]+1)))
    F0 = jnp.sqrt(_VAR_NORM * var_r * (1.0 - phi**2) / denom)
    return F0 * shape # sigma_l, shape (L,)
```

Finally an inverse spherical-harmonic transform turns the coefficients $r_{\ell m}$ into a real field $r(\mathbf{x})$ on the grid, which enters (1). The pattern is clipped at $\pm 3\sigma$ and the multiplicative factor $1+\mu r$ is clipped to $[0.1, 1.9]$, both standard safeguards from Palmer et al. (2009) that keep a rare large draw from flipping the sign of a tendency.

**An honest caveat about $\sigma$ (`sppt.py`).** The normalisation constant $C$ is calibrated *once*, at T21. The transform convention is not exactly resolution-independent, so at T42 and finer the true grid-point standard deviation is about $1.12 \times \sigma$. This does *not* affect anything in this tutorial: parameter recovery is done entirely at T21, and even at other resolutions the constant cancels between the "truth" model and the forecast model (they share the code). Only the *physical interpretation* of a learned $\sigma$ carries the offset. We leave it constant deliberately, to watch how the optimiser tunes the *effective* $\sigma$.

The three parameters are stored in **log space**, so that gradient steps can never make them non-physical (negative), and so that a multiplicative change is a fixed additive step regardless of scale:

```
class SPPTParams:
    log_sigma # sigma = exp(log_sigma), grid-point std of r (default 0.5)
    log_tau # tau = exp(log_tau), decorrelation time (s) (default 6 h)
    log_len # ell = exp(log_len), correlation length (m) (default 500 km)
```

# 4 Generating and storing the "truth"

To *learn* the parameters we need something to score the forecast against. This tutorial uses the cleanest possible setup, an **identical-twin** experiment (Mode A, milestone M1):

1. Pick a set of initial conditions ("cases"). We spin the deterministic model up for 120–200 days to reach a statistically steady midlatitude flow, then sample $N_c$ atmospheric states a few days apart. Each sampled state is one case's initial condition.
2. For each case, run the *stochastic* model forward with a *known* set of parameters $\theta^\star = (\sigma = 0.5,\ \tau = 6\,\mathrm{h},\ \ell = 500\,\mathrm{km})$ and save the state at each lead time. That saved trajectory is the "truth" $y$ for that case.
3. Start the optimiser from a deliberately *wrong* guess and check that it recovers $\theta^\star$.

Because the "truth" is generated by the *same model* we will forecast with, there is *no model error* — only the parameters differ. This isolates the learning machinery. (Mode B, where the truth is a higher-resolution run and SPPT must compensate genuine model error, is the harder, later milestone.)

**What is stored, and in what shape.** A dataset (`generate_data.py`) is a dictionary saved with `pickle`:

```
{
  "ic_specs": SpectralState, # the N_c initial conditions (spectral)
  "truth": {field: array[N_c, n_lead, lat, lon]}, # 5 verification fields
  "lead_steps": (steps_per_lead, ...),
  "forecast_resolution": "T21", "n_lev": 20,
}
```

The five verification fields are `u850`, `v850` (winds at 850 hPa), `t500` (temperature at 500 hPa), `vort500` (relative vorticity at 500 hPa) and `ps` (surface pressure) — a small, standard set spanning the mass and wind fields at low and mid levels. Each is diagnosed from the spectral state by transforming to the grid and interpolating in log-pressure (`diagnostics.py`). The initial conditions are stored in spectral space (they are the exact model state); the truth is stored as grid-space verification fields (that is what we score).

**Design question (raised by a careful reader): why only *one* stochastic run per initial condition? If I ran the model several times from the same IC and stored each as a separate target, would training improve — and at what cost?**

*Short answer: yes, it would reduce the small bias discussed in §7.4, it is cheap at training time, and it costs storage and truth-generation time linear in the number of draws. Here is why.*

For a fixed initial condition the "truth" is itself a *random* draw $y \sim F^\star$ from the true forecast distribution (the stochastic model with $\theta^\star$). The quantity we would *like* to minimise is the expected score $\mathbb{E}_{y\sim F^\star}[\,\mathrm{CRPS}(F_\theta, y)\,]$. With a single stored truth draw we replace that expectation by a one-sample estimate $\mathrm{CRPS}(F_\theta, y_1)$, which is noisy and slightly biased. If instead we store $K$ independent truth draws $y_1, \ldots, y_K$ from the same IC and average,

$$\frac{1}{K}\sum_{k=1}^{K} \mathrm{CRPS}(F_\theta, y_k) \xrightarrow[K\to\infty]{} \mathbb{E}_{y\sim F^\star}[\mathrm{CRPS}(F_\theta, y)], \tag{6}$$

the estimate's variance falls like $1/K$ and its minimiser moves towards $\theta^\star$.

**Cost.** The *expensive* object in a training step is the forecast ensemble rollout $F_\theta$ (a reverse-mode-differentiated integration; see §7.3). That rollout is computed *once per case* and can be scored against all $K$ truth draws — the extra $K-1$ comparisons are just cheap CRPS evaluations ($O(M^2)$ arithmetic on already-computed fields), *no extra model integrations*. So more truth draws barely change the per-step training time. What they *do* cost is (i) truth-generation time, which is $K$ forward integrations per case, and (ii) storage, $K\times$ the dataset size. Contrast this with adding more *cases* (distinct ICs), which samples different weather regimes and reduces a different component of the variance, or adding more *ensemble members* $M$, which is expensive because it multiplies the reverse-mode rollout cost. **Exercise 1** asks you to implement multi-draw truth and measure the effect.

**A known imperfection in Mode A truth.** The truth generator re-initialises the SPPT pattern at each lead boundary rather than carrying it continuously. At $\tau = 6\,\mathrm{h}$ with multi-day leads this is numerically inert ($\phi^n \sim 10^{-4}$ between leads — the pattern has fully decorrelated, so a fresh stationary draw is statistically identical to a carried one). It would matter for long $\tau$ or sub-daily leads, and is flagged in the log as a follow-up.

## 5 Scoring an ensemble: the CRPS

We need a single number measuring how well an ensemble forecast matches a scalar truth $y$, such that the number is minimised (in expectation) exactly when the forecast distribution equals the true one. Such a **proper scoring rule** is the *Continuous Ranked Probability Score* (Gneiting & Raftery, 2007). For a predictive distribution with CDF $F$ and an observation $y$,

$$\mathrm{CRPS}(F, y) = \int_{-\infty}^{\infty} \big(F(x) - \mathbf{1}\{x \geq y\}\big)^2 \, dx \;=\; \mathbb{E}_F|X - y| - \tfrac{1}{2}\,\mathbb{E}_F|X - X'|, \tag{7}$$

where $X, X'$ are independent draws from $F$. The second form is the useful one for ensembles: the first term rewards putting mass near $y$; the second term ($-\frac{1}{2}$ mean pairwise distance) rewards *sharpness* but penalises overconfidence, and it is what makes CRPS proper — it is minimised when $F = F^\star$.

**The finite-ensemble bias, and the "fair" fix.** With only $M$ members $x_1, \ldots, x_M$ the natural plug-in estimator uses $\frac{1}{M^2}\sum_{i,j}|x_i - x_j|$ for the second term. This *under*-estimates the pairwise spread of the underlying distribution (it includes the $M$ zero terms $|x_i - x_i|$ and uses the biased $1/M^2$ normaliser), so it *rewards* an ensemble that is too narrow: a small ensemble looks artificially good. The **fair CRPS** (Leutbecher, 2018; Lang et al., 2024) removes this by using the unbiased pairwise estimator $\frac{1}{M(M-1)}\sum_{i\neq j}$:

$$\mathrm{fCRPS} = \frac{1}{M}\sum_{i=1}^{M}|x_i - y| \;-\; \frac{1}{2M(M-1)}\sum_{i=1}^{M}\sum_{j=1}^{M}|x_i - x_j|. \tag{8}$$

Fair CRPS estimates the score the *infinite* ensemble would get, so it does not reward under-dispersion; it is the correct target when training an ensemble system to be reliable. The **almost-fair** variant (Lang et al., 2024) interpolates with a parameter $\alpha \in [0, 1]$,

$$\mathrm{afCRPS} = \frac{1}{M}\sum_{i}|x_i - y| \;-\; \frac{1-\epsilon}{2M(M-1)}\sum_{i,j}|x_i - x_j|, \qquad \epsilon = \frac{1-\alpha}{M}, \tag{9}$$

with $\alpha = 1$ giving fair CRPS. The small $\epsilon$ avoids a degeneracy Lang et al. identified when training (it keeps a slight preference for finite spread, which stabilises early optimisation). We use $\alpha = 0.95$. In code (`metrics.py`), vectorised over grid points:

```python
def afcrps(ensemble, truth, alpha=0.95): # ensemble: (M, ...), truth: (...)
    M = ensemble.shape[0]
    eps = (1.0 - alpha) / M
    e1 = jnp.mean(jnp.abs(ensemble - truth[None]), axis=0) # skill term
    diff = jnp.abs(ensemble[:, None] - ensemble[None, :]) # (M, M, ...)
    e2 = jnp.sum(diff, axis=(0, 1)) / (2.0 * M * (M - 1)) # fair spread term
    return jnp.mean(e1 - (1.0 - eps) * e2)
```

The training loss aggregates afCRPS over cases, lead times, and the five fields, each field normalised by its truth standard deviation so that surface pressure (in Pa, $O(10^3)$) does not dominate temperature (in K, $O(10)$):

$$J(\theta) = \frac{1}{5} \sum_{f \in \text{fields}} \frac{1}{s_f} \frac{1}{N_c N_\ell} \sum_{c,\lambda} \text{afCRPS}\big(F_\theta^{c,\lambda,f},\, y^{c,\lambda,f}\big), \qquad s_f = \text{std}(y^f). \tag{10}$$

# 6 Is the ensemble calibrated? Two diagnostics

A low score is necessary but not transparent. Two classical diagnostics *visualise* reliability directly.

## 6.1 Spread–error ratio

Intuitively, a reliable ensemble's spread should match the error of its mean: if the members scatter with standard deviation $s$, the truth should typically sit about $s$ away from the ensemble mean. The precise statement carries a finite-ensemble correction that is easy to derive.

Suppose the ensemble is *reliable* in the strong sense that the truth $y$ and the $M$ members $x_1, \ldots, x_M$ are **exchangeable** — all $M{+}1$ are i.i.d. draws from a common distribution with variance $\sigma_c^2$ about a common mean. Write $\bar{x} = \frac{1}{M}\sum_i x_i$ and the sample variance $s^2 = \frac{1}{M-1}\sum_i (x_i - \bar{x})^2$. Then

$$\mathbb{E}[s^2] = \sigma_c^2, \tag{11}$$

$$\mathbb{E}\big[(y - \bar{x})^2\big] = \text{Var}(y) + \text{Var}(\bar{x}) = \sigma_c^2 + \frac{\sigma_c^2}{M} = \frac{M+1}{M}\,\sigma_c^2, \tag{12}$$

using independence of $y$ from the members and $\text{Var}(\bar{x}) = \sigma_c^2/M$. Dividing (12) by (11),

$$\mathbb{E}\big[(y - \bar{x})^2\big] = \frac{M+1}{M}\,\mathbb{E}[s^2] \quad \Longrightarrow \quad \boxed{\text{ratio} \;=\; \sqrt{\tfrac{M+1}{M}}\,\frac{\text{spread}}{\text{rmse}} \;=\; 1} \tag{13}$$

for a reliable ensemble (Fortin et al., 2014; Leutbecher, 2018), where spread $= \sqrt{\mathbb{E}[s^2]}$ (root-mean ensemble variance) and rmse $= \sqrt{\mathbb{E}[(y-\bar{x})^2]}$ (root-mean-square error of the mean). The $\sqrt{(M{+}1)/M}$ factor is the correction for the fact that a finite ensemble mean is itself uncertain; forgetting it makes every finite ensemble look slightly under-dispersed. A ratio *below* 1 means the ensemble is **under**-dispersed (overconfident: errors exceed the spread); *above* 1 means **over**-dispersed. The code (`metrics.py`) pools the expectations over cases and grid points:

```python
def spread_error_ratio(ensemble, truth): # -> 1 for a reliable ensemble
    M = ensemble.shape[0]
    return jnp.sqrt((M + 1.0) / M) * spread(ensemble) / rmse_of_mean(ensemble, truth)
```

**Validate the diagnostic, not just the code.** The first version of this diagnostic (and its unit test) used a wrong formula that nonetheless *passed* against a mis-constructed test scenario. The fix was to build a genuinely exchangeable ensemble in the test (truth and

members all i.i.d. from one centre) and confirm the ratio is $\approx 1$ (measured 1.013). Lesson: a test that passes against the wrong scenario proves nothing — check that the scenario itself encodes the property you are testing.

### 6.2 Rank histogram (Talagrand diagram)

The rank histogram (Hamill, 2001) checks reliability without assuming Gaussianity. For each verification point, pool the $M$ members and the truth, sort them, and record the **rank** of the truth among the members: the number of members below it, an integer in $\{0, 1, \ldots, M\}$ ($M{+}1$ possible values). If the truth is statistically indistinguishable from a member (the ideal), it is equally likely to fall in any of the $M{+}1$ gaps, so **the histogram of ranks over many points is flat**. Departures from flat have textbook shapes:

Table 2: Reading a rank histogram.

| shape | interpretation |
|---|---|
| flat (uniform) | calibrated — truth behaves like a member |
| $\cup$ (U-shaped) | **under**-dispersed: truth too often an outlier (rank 0 or $M$) |
| $\cap$ (dome) | **over**-dispersed: truth too often central |
| sloped / asymmetric | **biased**: forecast systematically off to one side |

```
def rank_histogram(ensemble, truth): # length M+1
    rank = jnp.sum(ensemble < truth[None], axis=0).ravel() # 0..M per point
    return jnp.bincount(rank, length=ensemble.shape[0] + 1)
```

A third, informal check — overlaying the histogram of the members' anomalies (a member minus the ensemble mean) with the truth's anomaly — shows directly whether the truth "looks like" a draw from the ensemble.

## 7 Learning the parameters: differentiating through randomness

This is the conceptual core. We want $\nabla_\theta J(\theta)$, the gradient of the ensemble score (10) with respect to $\theta = (\log\sigma, \log\tau, \log\ell)$. But $J$ is defined through a *random* object — the ensemble depends on the noise draws. What does it even mean to differentiate through randomness, and what is the derivative taken with respect to?

### 7.1 The reparameterization (pathwise) gradient

Write the noise used by one ensemble member as $\eta$ — the whole collection of i.i.d. standard Gaussians drawn for the stationary initialisation and every AR(1) step (2). The key observation is that in (2) the noise $\eta$ is drawn from a **fixed** distribution ($\mathcal{N}(0,1)$) that *does not depend on* $\theta$; all the $\theta$-dependence lives in the *coefficients* $\phi(\theta)$ and $\sigma_\ell(\theta)$. Therefore a member's entire trajectory — and hence each of its five verification fields — is a **deterministic, differentiable function** of $\theta$ once $\eta$ is fixed:

$$x_\theta^{(m)} = G(\theta, \eta^{(m)}), \qquad \eta^{(m)} \sim p(\eta) \text{ independent of } \theta. \tag{14}$$

This is the **reparameterization trick** (also "pathwise" gradient) (Kingma & Welling, 2014; Mohamed et al., 2020): instead of sampling $x$ from a $\theta$-dependent distribution (which we could not differentiate directly), we sample $\theta$-independent noise $\eta$ and pass it through a differentiable map $G$. The loss becomes an expectation over the *fixed* noise law,

$$J(\theta) = \mathbb{E}_{\eta^{(1:M)}\sim p}\Big[\,\mathrm{afCRPS}\big(\{G(\theta,\eta^{(m)})\}_{m=1}^{M},\ y\big)\Big], \tag{15}$$

and because the distribution we average over no longer depends on $\theta$, the gradient passes *inside* the expectation:

$$\nabla_\theta J = \mathbb{E}_{\eta^{(1:M)}}\Big[\,\nabla_\theta \,\mathrm{afCRPS}\big(\{G(\theta,\eta^{(m)})\},y\big)\Big]. \tag{16}$$

We estimate the expectation (16) by **Monte Carlo**: draw one batch of cases and one set of member noises, and evaluate the bracket. That is a *stochastic gradient* — an unbiased, noisy estimate of $\nabla_\theta J$ — exactly the kind stochastic optimisers are built for.

**What is the gradient with respect to?** With respect to the three *log-parameters* $\theta = (\log\sigma, \log\tau, \log\ell)$ — numbers, not fields. **What flows through it?** Reverse-mode automatic differentiation (`jax.grad`) traces the dependence of the score on $\theta$ backwards through the afCRPS, through every member's multi-day integration (dynamics + SPPT), and into $\phi(\theta)$ and $\sigma_\ell(\theta)$. **What is held fixed?** The noise $\eta$ (the $\mathcal{N}(0,1)$ draws) and the truth $y$: $y$ is data, so no gradient flows into it. The afCRPS is built from absolute values $|x-y|$, which are differentiable everywhere except on a measure-zero set, so the pathwise gradient is well defined almost surely.

### 7.2 Common random numbers

The estimator (16) is unbiased for *any* way of drawing the $\eta$. But if we re-drew fresh noise every optimiser step, two nearby $\theta$'s would be scored with different noise, and the difference in their losses would be dominated by noise rather than by the change in $\theta$ — a very high-variance gradient. We therefore use **common random numbers**: within a single gradient evaluation the member noises are fixed (derived deterministically from a per-step key). This is the Monte Carlo analogue of comparing two policies on the *same* random scenarios, and it dramatically reduces the variance of the stochastic gradient. It is essential for optimising a stochastic scheme in practice.

```
def member_keys(base_key, case_idx, n_members): # deterministic per (case, member)
    return jax.random.split(jax.random.fold_in(base_key, case_idx), n_members)

# one training step (train.py, condensed):
grad_fn = jax.jit(jax.value_and_grad(loss_fn)) # compile once
for step in range(n_steps):
    idx = choose_a_random_batch_of_cases(step)
    eval_key = fold_in(base_key, 10_000 + step) # fixed WITHIN this step -> CRN
    loss, grads = grad_fn(params, ic_specs, truth, idx, ..., eval_key, ...)
    updates, opt_state = optax.adam(lr).update(grads, opt_state)
    params = optax.apply_updates(params, updates) # gradient DESCENT on theta
```

### 7.3 Reverse mode, checkpointing, and why we train at T21

Reverse-mode autodiff must store intermediate states of the forward integration to replay them backwards. A single T42 spectral state is ∼10 MB; a 10-day rollout is ∼720 steps, so one differentiated trajectory needs ∼7.5 GB — *per member, per case*. Multiplying by an ensemble batch overflows an 18 GB laptop. Two mitigations are used and one decision is forced:

- **Gradient checkpointing** (`jax.checkpoint` around the scan body in `rollout.py`) stores only a subset of states and recomputes the rest during the backward pass — trading compute for memory.
- **Vectorising over members with** `vmap` and compiling once with `jax.jit` keeps the step efficient (compile the graph once, then pay only arithmetic per step).

- **Resolution.** Even so, differentiable training at T42 is infeasible on the laptop, so the milestone-M1 sweep is done at **T21**, where the memory is light. Forward-only diagnostics (no gradient, no stored trajectory) are cheap at any resolution. Real T42/T127 *training* belongs on a GPU.

**Reproducibility engineering.** On this machine any single process is killed after ∼10 minutes, and the reverse-mode compile of a 5-day-lead graph alone takes ∼8.5 minutes. Two consequences shaped the runnable pipeline: (i) the recovery sweep uses a **1-day lead** (48 T21 steps), whose graph compiles in seconds — and since $\sigma$-recovery only needs the ensemble spread to match the identical-twin truth's spread, the short lead is faithful; (ii) the sweep **checkpoints every optimiser step to disk** under a wall-clock budget, so an interrupted run is resumed by simply re-invoking it. Heavy `JAX` compute runs in the foreground; plotting is a separate pure-`numpy` step.

### 7.4 Why $\sigma$ settles slightly *above* 0.5

A natural worry: *will $\sigma$ really converge to the true 0.5?* The honest answer is: to *near* 0.5, settling a couple of percent *above*, and here is exactly why. Fair CRPS is a *strictly proper* scoring rule, so the population objective $\mathbb{E}_{y\sim F^\star}[\mathrm{fCRPS}(F_\theta, y)]$ is minimised at $F_\theta = F^\star$, i.e. at $\theta = \theta^\star$ — *provided* the forecast family contains the truth (it does, identical twin) and we optimise the true expectation. We do not optimise the true expectation; we optimise a finite sample of it (10). Three finite-sample effects act:

1. **Finite members** $M$: removed to first order by the fairness correction (8). This is the one we *do* fix exactly.
2. **Finite cases** $N_c$: the sample-average objective is a random function whose minimiser scatters around $\theta^\star$.
3. **One truth draw per case**: each case contributes $\mathrm{CRPS}(F_\theta, y_c)$ for a *single* random $y_c$. To keep the score low on an outlying $y_c$, the optimiser slightly *inflates* the spread — a small *upward* bias in $\sigma$.

Effects 2 and 3 are *sampling* properties, not bugs, and both shrink as $N_c$ and the number of truth draws grow (§4, Exercise 1). In our run $\sigma$ plateaus at $\approx 0.508$ — about 2% high — entirely consistent with this account.

## 8 Experimental protocol

**Recovery sweep (training).** T21 identical-twin dataset, seed 0, $N_c = 12$ cases, truth generated with $\theta^\star = (\sigma{=}0.5, \tau{=}6\mathrm{h}, \ell{=}500\mathrm{km})$, 1-day lead. We recover $\sigma$ alone (freezing $\tau, \ell$ at truth by masking their gradients) — the single-parameter problem gives the cleanest, most interpretable convergence story. Optimiser: Adam (Kingma & Ba, 2015), learning rate 0.03; batch of 3 cases per step, $M = 4$ members; start from a deliberately wrong $\sigma = 0.2$.

**Calibration check (independent evaluation).** A *separate*, freshly generated identical-twin dataset with a **different random seed** (seed 1), so the truth draws and the sampled weather states are independent of the training set. Forward-only ensembles ($M = 8$, no gradient) at the *recovered* $\sigma$, at 3- and 5-day leads. The spread–error ratio and rank histogram are computed on this held-out data. This is our train/validation split: parameters are fitted on seed 0 and their calibration is judged on unseen seed-1 weather. There is no separate hyperparameter-tuning set — the learning rate and $\alpha$ were fixed by hand (Exercise 2 suggests a proper split).

**On learning-rate choice.** An early run with lr = 0.1 let Adam's momentum carry $\sigma$ past 0.5 to $\approx 0.73$ before turning around — an *overshoot*, distinct from the finite-sample bias. Dropping to lr = 0.03 removes the overshoot and gives the clean monotone approach shown below. The two knobs are orthogonal: learning rate controls overshoot, number of cases/draws controls the residual bias.

# 9 Results — Milestone M1

## 9.1 Parameter recovery

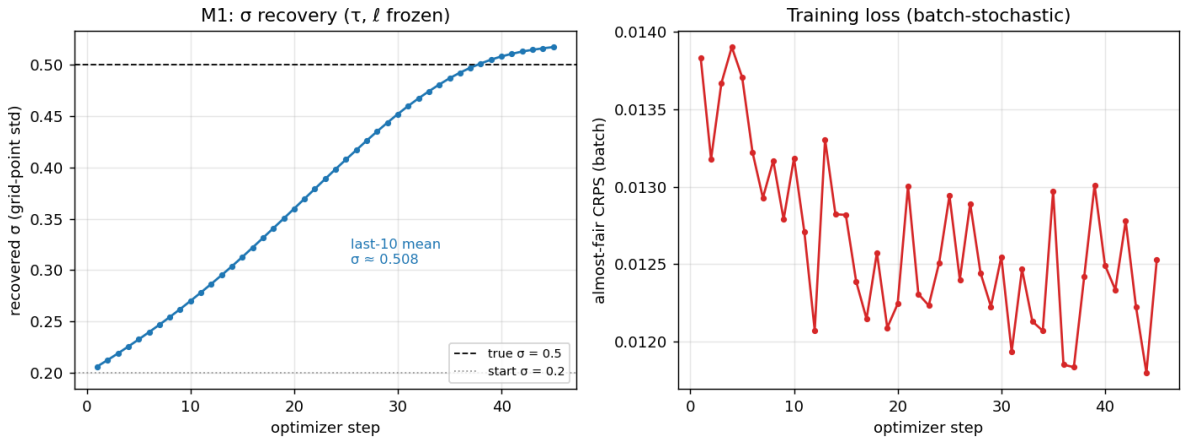


Figure 1: **Left:** the recovered $\sigma$ versus optimiser step. Starting from the wrong value 0.2 (dotted), $\sigma$ rises along a smooth, decelerating, monotone curve, crosses the true 0.5 (dashed) near step 38, and plateaus at $\approx 0.51$ (mean of the last ten steps = 0.508; final value 0.517) with no overshoot. **Right:** the training loss. It is *batch-stochastic* — each step scores a different random 3-case batch with its own common-random-number draw — so it wobbles and must *not* be read as a smooth descent. The parameter trajectory, not the raw loss, is the convergence signal.

Figure 1 is the headline evidence that the learning machinery works: the gradient of an ensemble score, taken through a multi-day stochastic integration by reverse-mode autodiff, points in the right direction and Adam walks $\sigma$ from 0.2 to the neighbourhood of the truth. The $\approx 2\%$ residual is the finite-sample bias of §7.4, not an error.

## 9.2 The recovered $\sigma$ gives a calibrated ensemble

Table 3: Spread–error ratios behind Figure 2 (target = 1).

| lead | u850 | v850 | t500 | vort500 | ps |
|---|---|---|---|---|---|
| 3 d | 1.045 | 0.971 | 0.924 | 0.998 | 0.971 |
| 5 d | 1.001 | 0.966 | 0.917 | 0.947 | 0.960 |

Taken together, Figures 1–4 confirm milestone M1 on two fronts: the optimiser *recovers* the parameter, and the recovered parameter *produces a calibrated ensemble* on independent data.

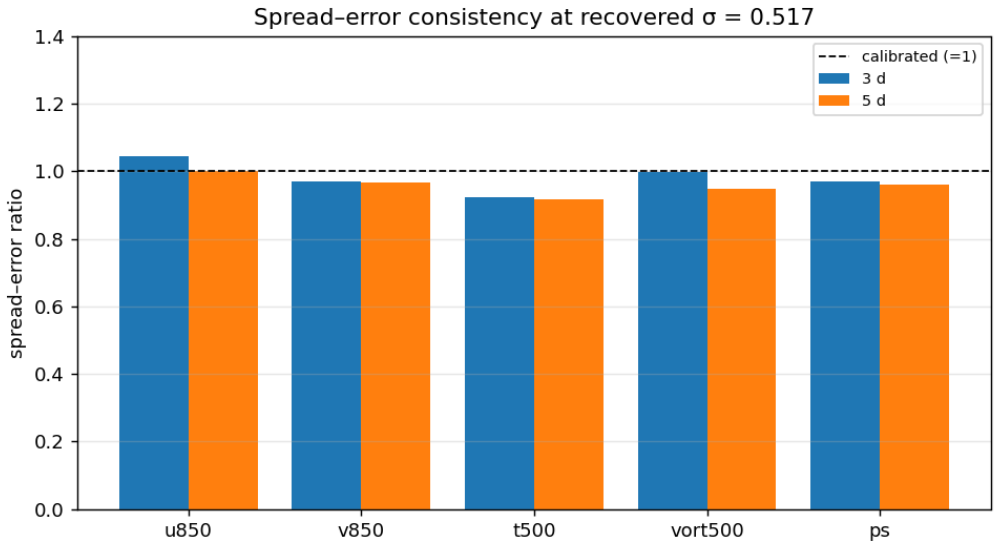


Figure 2: Spread–error ratio (13) at the recovered $\sigma = 0.517$, for the five verification fields at 3- and 5-day leads, on the independent seed-1 dataset. All values lie in $[0.92, 1.05]$; the dashed line is the calibrated target 1. The ensemble is very slightly *under*-dispersed for `t500` ($\approx 0.92$) — consistent with $\sigma$ tuned at a 1-day lead being a hair low for the mid-tropospheric temperature spread at 3–5 days — but is within the calibrated band for every field.

## 10 How the figures were made

Every figure is reproduced by three small scripts in `examples/stochastic_sppt/figures/` (all forcing float64+CPU at import):

`run_sweep.py` builds the T21 identical-twin training dataset (cached to `_artifacts/`), then runs the $\sigma$-only recovery sweep. It freezes $\tau, \ell$ by *masking* their gradients, checkpoints `{step, params, opt_state, history}` every step, and honours a wall-clock budget so it is resumable:

```
def _sigma_only(grads): # keep only the log_sigma gradient
    return grads.replace(log_tau=jnp.zeros_like(grads.log_tau),
                         log_len=jnp.zeros_like(grads.log_len))
# ... each step: loss, grads = grad_fn(...); grads = _sigma_only(grads); adam.
    update(...)
# ... pickle {step, params, opt_state, history} to _artifacts/sweep_ckpt.pkl
```

`run_calibration.py` generates a fresh seed-1 dataset and runs forward-only ensembles at the recovered $\sigma$ (read from the sweep checkpoint), computing the spread–error ratios, rank histogram, and anomaly distributions, saved to `calib_results.pkl`.

`plot.py` is pure `numpy`/`matplotlib` — no `JAX` — and renders all five PNGs from the two saved artifacts. Separating slow compute from fast plotting means a figure can be re-styled in a second.

To reproduce from scratch (from the repository root):

```
python -m examples.stochastic_sppt.figures.run_sweep --data-only
python -m examples.stochastic_sppt.figures.run_calibration --data-only
# run the sweep; re-invoke until it prints DONE (resumes from its checkpoint):
python -m examples.stochastic_sppt.figures.run_sweep --target-steps 45 --budget-
    seconds 520
```

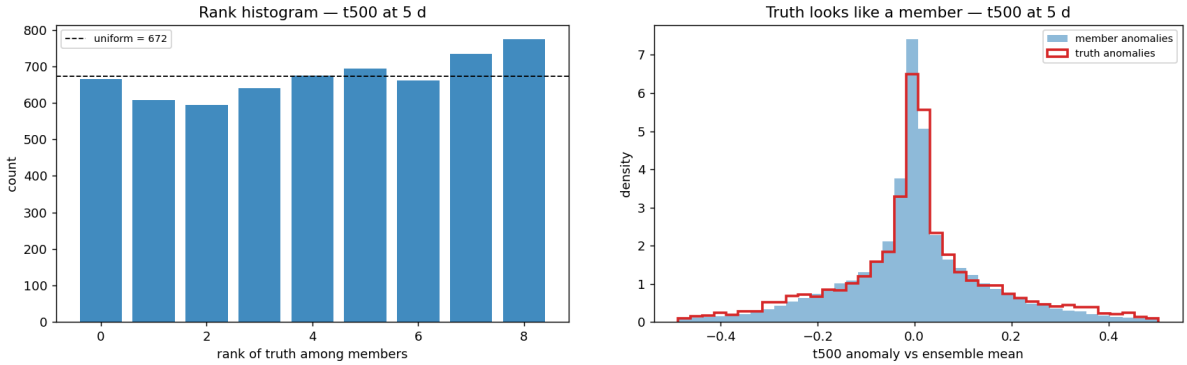


Figure 3: **Left:** rank histogram of `t500` at 5 days (uniform level $\approx 672$ per bin). It is approximately flat with a mild upward tilt towards the higher ranks — the same slight under-dispersion (and a hint of asymmetry) seen in Table 3 — but far from the deep $\cup$ shape of a badly under-dispersed ensemble. **Right:** the distribution of the members' `t500` anomalies (filled) with the truth's anomalies overlaid (outline). The truth is a touch more peaked but tracks the member distribution closely: the truth looks like another draw from the ensemble.

```
python -m examples.stochastic_sppt.figures.run_calibration
python -m examples.stochastic_sppt.figures.plot
```

## 11 Limitations and open questions

- **Only $\sigma$ was recovered.** Recovering $\tau$ and $\ell$ jointly is harder: $\tau$ and $\ell$ are more weakly identified by a short-lead score than $\sigma$ is, and the three interact through $F_0$ (5). This is the obvious next experiment.
- **Mode B (real model error)** — training SPPT so a coarse ensemble covers the error against a higher-resolution truth — is the milestone that matters for applications, and needs a GPU/cluster.
- **Resolution dependence of $\sigma$** (§3) should be removed if a learned $\sigma$ is to be read as a physical grid-point standard deviation across resolutions.
- **The finite-sample bias** in $\sigma$ is understood (§7.4) but not eliminated; multi-draw truth is the remedy.

## 12 Exercises

1. **Multi-draw truth.** Modify `generate_data.py` to store $K$ independent truth draws per case, and average the afCRPS over them in the loss. Measure the recovered $\sigma$ and its scatter as a function of $K \in \{1, 2, 4, 8\}$. Confirm the plateau moves towards 0.5 and quantify the extra data-generation and per-step training cost — verifying the claim in §4 that truth draws are cheap at training time.
2. **A proper split.** Hold out a third dataset (seed 2) as a true test set; use seed 1 only to choose the learning rate and $\alpha$; report the final calibration on seed 2. Does the conclusion change?
3. **Joint recovery.** Unfreeze $\tau$ and $\ell$. Start from wrong values for all three and see which are recovered and which are not. Relate identifiability to lead time (try a longer lead if you have a GPU).

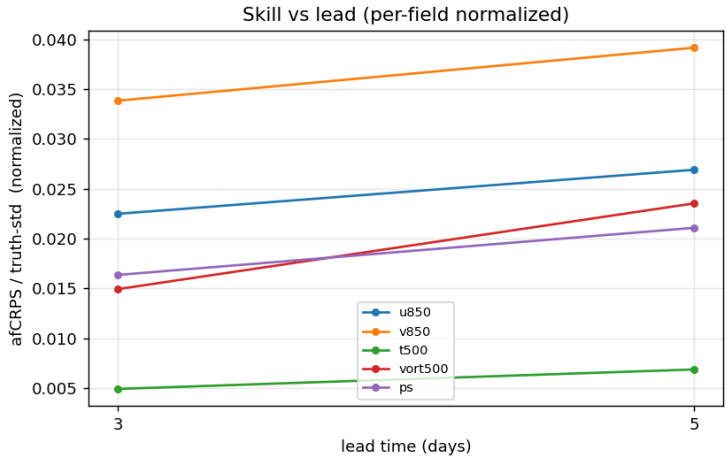


Figure 4: afCRPS versus lead time, *normalised per field by the truth standard deviation* (the scaling used in the training loss, and the fix to an earlier panel that plotted the score unnormalised and was visually dominated by surface pressure in Pa). Skill degrades gracefully from 3 to 5 days, as expected.

4. **Overshoot vs. bias.** Reproduce the lr $= 0.1$ overshoot and confirm it is momentum, not sampling, by turning momentum off (plain SGD).

5. **Break the fairness correction.** Replace (8) with the naive $1/M^2$ estimator and watch the ensemble collapse (recovered $\sigma$ far too small) — a hands-on demonstration of why fair CRPS matters.

## Further reading

- **SPPT and the pattern generator:** Palmer et al. (2009) (the scheme is reproduced from its Appendix 8.1); Leutbecher et al. (2017) for the modern SPPT/SKEB state of the art.
- **Scoring rules:** Gneiting & Raftery (2007) (proper scoring rules, the definitive reference); Hersbach (2000) (CRPS for ensemble systems); Leutbecher (2018) and Lang et al. (2024) (fair / almost-fair CRPS).
- **Calibration diagnostics:** Hamill (2001) (rank histograms and their pitfalls); Fortin et al. (2014) (the spread–error relation done correctly).
- **Stochastic gradients:** Mohamed et al. (2020) (a superb survey of Monte-Carlo gradient estimation, including the reparameterization vs. score-function distinction); Kingma & Welling (2014) (reparameterization in context); Kingma & Ba (2015) (Adam).
- **The idealised model:** Held & Suarez (1994).

## References

Fortin, V., Abaza, M., Anctil, F., & Turcotte, R. (2014). Why should ensemble spread match the RMSE of the ensemble mean? *Journal of Hydrometeorology*, 15(4), 1708–1713.

Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359–378.

Hamill, T. M. (2001). Interpretation of rank histograms for verifying ensemble forecasts. *Monthly Weather Review*, 129(3), 550–560.

Held, I. M., & Suarez, M. J. (1994). A proposal for the intercomparison of the dynamical cores of atmospheric general circulation models. *Bulletin of the AMS*, 75(10), 1825–1830.

Hersbach, H. (2000). Decomposition of the continuous ranked probability score for ensemble prediction systems. *Weather and Forecasting*, 15(5), 559–570.

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. *ICLR*. (The reparameterization trick.)

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*.

Lang, S., et al. (2024). AIFS-CRPS: ensemble forecasting using a model trained with an almost-fair CRPS. *npj Climate and Atmospheric Science / arXiv:2412.15832*.

Leutbecher, M., et al. (2017). Stochastic representations of model uncertainties at ECMWF: state of the art and future vision. *Quarterly Journal of the RMS*, 143(707), 2315–2339.

Leutbecher, M. (2019). Ensemble size: How suboptimal is less than infinity? *Quarterly Journal of the RMS*, 145, 107–128. (Fair scores for finite ensembles.)

Mohamed, S., Rosca, M., Figurnov, M., & Mnih, A. (2020). Monte Carlo gradient estimation in machine learning. *Journal of Machine Learning Research*, 21(132), 1–62.

Palmer, T. N., Buizza, R., Doblas-Reyes, F., Jung, T., Leutbecher, M., Shutts, G. J., Steinheimer, M., & Weisheimer, A. (2009). Stochastic parametrization and model uncertainty. *ECMWF Technical Memorandum* 598.